Modelado de Datos con Estructuras (struct) y Punteros

Definición de una entidad de proyecto y carga por dirección

Alexis Maier

E.E.S.T. N° 1 "Eduardo Ader" — Vicente López

Laboratorio de Programación (LPR) — 5° Año 3° División — Grupos A-B

Profesor: Prof. Mansilla Muñoz York Elías

Fecha: 01 de octubre de 2026

Introducción

La actividad 5 consolidó el modelado de datos mediante estructuras, el mecanismo con el que C++ agrupa variables heterogéneas bajo un mismo nombre para representar una entidad con estado. La consigna proponía definir una estructura de proyecto denominada *EntidadProyecto* que representara el modelo de datos del Proyecto Integrador Anual, y cargarla desde una función auxiliar mediante pasaje por dirección, comprobando luego que los valores quedaran efectivamente almacenados en la memoria. La solución integra los conceptos presentados en las actividades previas: entrada por consola, punteros, cadenas de caracteres y modularización.

Desarrollo

Definición de la estructura

La estructura *EntidadProyecto* agrupa tres campos de tipos distintos bajo un único identificador: *int id* para el identificador numérico, *char nombre[50]* para el nombre o la descripción en texto, y *float metrica* para el valor decimal de operación, que puede representar lecturas de sensores, consumo o avance. Esa combinación de tipos es justamente lo que distingue una estructura de un simple arreglo: el arreglo es homogéneo y contiene un único tipo, mientras que la estructura modela un concepto con atributos diferentes.

Inicialización de seguridad

Antes de cargar los datos reales, la instancia se inicializa con *EntidadProyecto miEntidad = {0, "Sin cargar - LPR 5to", 0.0f};*. Esta práctica evita que la variable contenga valores residuales de la memoria, garantiza un comportamiento coherente ante un error previo y documenta el estado inicial de la entidad.

Pasaje por dirección

El prototipo *void cargarDatos(EntidadProyecto* ptr);* declara un puntero a la estructura. En *main()* la invocación se realiza con *cargarDatos(&miEntidad);*, donde el operador **&** entrega la dirección de la entidad. Dentro de la función los campos se alcanzan con el operador flecha: *ptr->id*, *ptr->nombre* y *ptr->metrica*. Al operar sobre la dirección, la función modifica directamente las celdas originales sin necesidad de devolver ningún valor.

Ingreso de datos y control del búfer

La lectura del identificador y de la métrica emplea *cin* >>, mientras que la captura del texto utiliza *cin.getline(ptr->nombre, 50)* precedida de *cin.ignore();* para limpiar el salto de línea residual del búfer. Cada campo se identifica con un mensaje en consola, lo que garantiza una interacción clara con el usuario.

Verificación posterior

Al regresar a *main()* se imprimen los tres campos leídos desde la propia variable *miEntidad*, usando el operador punto (*miEntidad.id*), lo que demuestra que el pasaje por dirección surtió efecto. Finalmente se muestra la dirección hexadecimal con *&miEntidad*, confirmando que la estructura ocupa una región contigua de memoria.

Tamaño y ocupación en memoria

En una arquitectura típica de 32 bits la estructura ocupa 4 bytes por el *int*, 50 por el arreglo de caracteres y 4 por el *float*, es decir 58 bytes útiles, ajustados a 60 por la alineación de memoria que exige el procesador. Este cálculo explica por qué el lenguaje conviene colocar el arreglo de caracteres en el medio de la estructura: evita un relleno adicional y reduce el desperdicio de RAM.

Cuestionario Técnico de Control

1. Definición de una estructura

Una estructura (*struct*) es un tipo de datos compuesto que agrupa variables relacionadas bajo un mismo nombre, cada una con su propio tipo. Permite modelar una entidad del mundo real, en este caso un proyecto con identificador, nombre y métrica, en lugar de manipular variables sueltas carentes de relación semántica.

2. Operador flecha frente a operador punto

Cuando se trabaja con la estructura directamente se usa el punto, como en *miEntidad.id*. Cuando se trabaja a través de un puntero se usa la flecha, como en *ptr->id*, que es azúcar sintáctico equivalente a *(*ptr).id*: primero se desreferencia el puntero para obtener la estructura y luego se accede al campo. Olvidar el asterisco dentro de los paréntesis es el error más frecuente al trabajar con punteros a estructuras.

3. Ventaja del pasaje por dirección

Pasar la dirección en lugar del valor evita copiar toda la estructura en memoria, lo que resulta especialmente relevante cuando la estructura contiene arreglos grandes. Además posibilita que la función modifique el dato original y no trabaje sobre una copia descartable al terminar la llamada.

4. sizeof y alineación

El operador *sizeof* devuelve el tamaño en bytes de un tipo o de una instancia. El resultado no siempre coincide con la suma de los campos, porque el procesador exige que ciertos tipos queden alineados en direcciones múltiplos de 4. Ese relleno se llama padding y, si se coloca un campo grande entre dos pequeños, el desperdicio crece.

5. Autoevaluación

La dificultad principal fue comprender la diferencia entre el punto y la flecha, ya que ambos parecen indicar acceso a un campo. Comprender que la flecha es una forma abreviada de combinar desreferencia y punto, y que su omisión produce errores de compilación con mensajes poco intuitivos, fue el avance decisivo de la actividad. Como mejora pendiente quedó ampliar el modelo con más campos y aplicar el taller completo a datos reales del Proyecto Integrador Anual.

Conclusión

La actividad 5 representa el punto de convergencia de los contenidos de la primera mitad del ciclo: estructuras, punteros, cadenas y modularidad dejan de ser técnicas aisladas y se combinan en un modelo de datos coherente. Comprender que una función puede recibir la dirección de una entidad y modificarla directamente supone un cambio de mentalidad: el programa deja de tratar las variables como números y pasa a operar sobre objetos con estado, paso previo a la programación orientada a objetos que desarrolla el Proyecto Integrador Anual. Las bases adquiridas permiten construir software capaz de representar mediciones de sensores y estado de módulos de hardware dentro del prototipo de Expotécnica.

Referencias

cppreference.com. (s. f.). *Struct*. Referencia del lenguaje C++. https://en.cppreference.com/w/cpp/language/struct

cplusplus.com. (s. f.). *Structures*. Tutorial de C++. https://cplusplus.com/doc/tutorial/structures/

Microsoft Learn. (s. f.). *Operadores de miembro > y ..* https://learn.microsoft.com/es-es/cpp/cpp/member-pointer-operators

Microsoft Learn. (s. f.). *Operador sizeof*. https://learn.microsoft.com/es-es/cpp/cpp/sizeof-operator